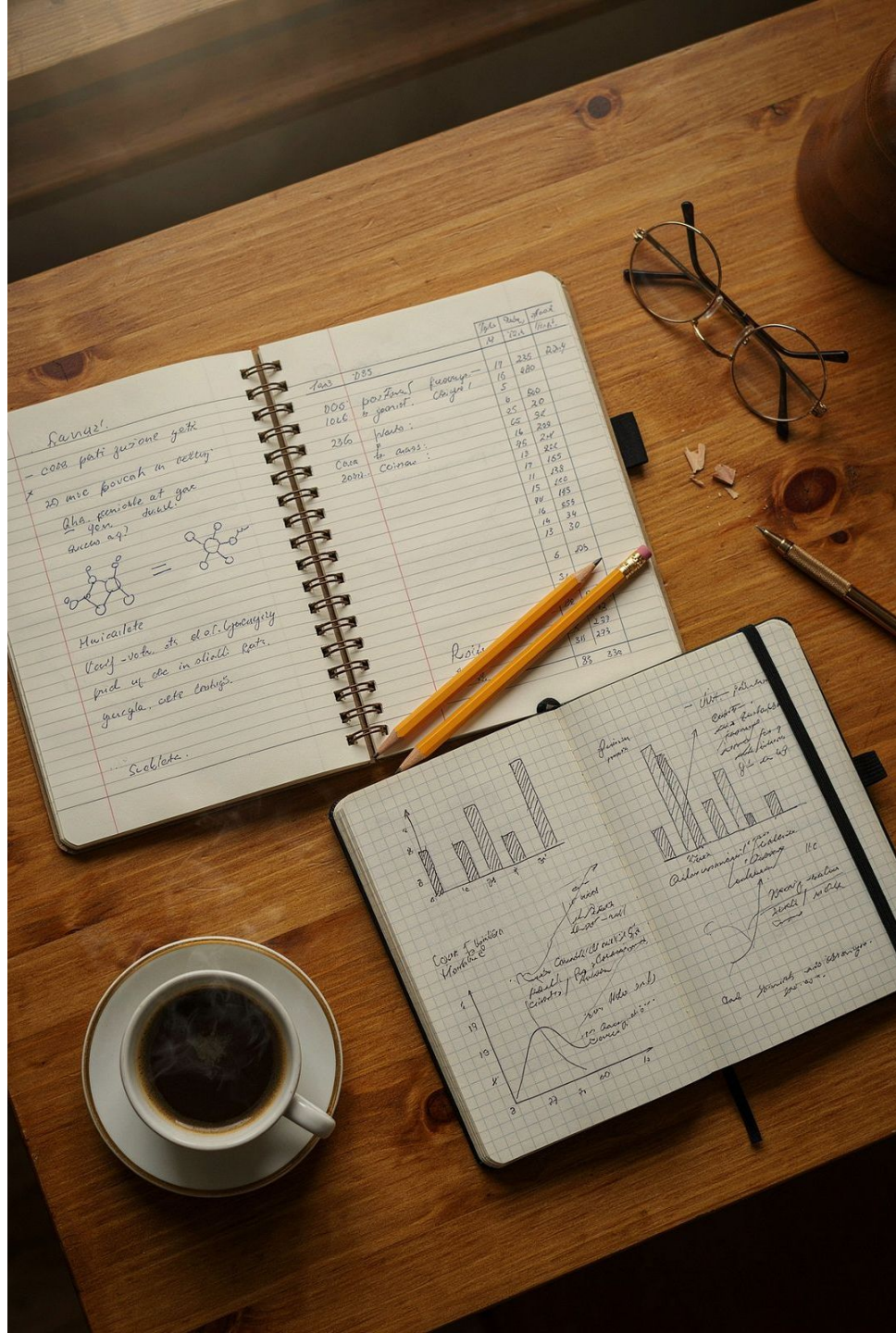
# Seguimiento de Experimentos en Machine Learning

Una guía práctica para registrar, versionar y reproducir tus experimentos de ML con rigor científico.

S15.1 · PÍLDORA FORMATIVA



# ¿Por qué registrar experimentos?

En ML es habitual lanzar docenas —o cientos— de experimentos antes de encontrar un modelo competitivo. Sin un registro sistemático, es imposible saber **qué configuración produjo qué resultado**, lo que conduce a trabajo duplicado, errores no detectados y resultados irrepetibles.

### 🔁 Reproducibilidad

Repetir exactamente un experimento pasado en cualquier momento

### 📊 Trazabilidad

Saber qué cambió entre dos versiones de un modelo

### 🤝 Colaboración

Compartir resultados verificables con el equipo

### ⚖️ Auditoría

Demostrar el proceso completo ante terceros o reguladores



# ¿Qué registrar en cada experimento?

Un registro completo cubre cuatro grandes categorías. La regla de oro es: **si cambias algo, regístralo**. Cuanto más metódico sea el registro, más fácil será depurar fallos y replicar éxitos.



### Hiperparámetros

Learning rate, batch size, número de capas, épocas, umbral de parada temprana...



### Métricas

Accuracy, F1, AUC-ROC, pérdida de entrenamiento y validación por época



### Artefactos

Pesos del modelo, gráficas, matrices de confusión, exportaciones ONNX



### Metadatos

Fecha, entorno, versión de librerías, hardware (GPU/CPU), autor

HERRAMIENTAS

# MLflow: seguimiento open-source

**MLflow** es la solución más extendida en entornos corporativos europeos. Ofrece una API Python sencilla para registrar experimentos y una interfaz web para explorarlos. Se puede desplegar localmente o en la nube sin coste de licencia.

### mlflow.log_param() / log_metric()

Registra hiperparámetros y métricas con una sola línea de código

### mlflow.log_artifact()

Guarda cualquier fichero como artefacto vinculado al experimento

### Model Registry

Gestiona el ciclo de vida del modelo: staging, producción, archivado



HERRAMIENTAS

# La opción mínima viable: un CSV bien estructurado

## ¿Cuándo usar un CSV?

Cuando el proyecto es pequeño, el equipo trabaja solo o las políticas de la organización impiden usar servicios en la nube. Un CSV es portátil, auditable y no requiere infraestructura.

La clave está en **definir las columnas desde el principio** y comprometerse a rellenarlas tras cada experimento, sin excepciones.

## Columnas recomendadas

| Campo | Ejemplo |
|---|---|
| experiment_id | exp_042 |
| fecha | 2024-11-15 |
| modelo | RandomForest |
| hiperparámetros | n_est=200, max_depth=10 |
| val_F1 | 0.847 |
| git_commit | a3f9c12 |
| notas | Aumenté dropout |

# Semillas y reproducibilidad estocástica

Los algoritmos de ML dependen de procesos aleatorios: inicialización de pesos, barajado de datos, dropout, splits train/val... Sin fijar la semilla aleatoria, dos ejecuciones del mismo código producirán resultados distintos.

### Fijar la semilla global

Establece `random.seed(42)`, `np.random.seed(42)` y `torch.manual_seed(42)` al inicio del script. En TensorFlow usa `tf.random.set_seed(42)`.

### Registrar la semilla como parámetro

Trata la semilla como un hiperparámetro más y guárdala en el registro del experimento. Así podrás reproducir exactamente los mismos splits y orden de batches.

### Cuidado con el hardware

Las operaciones en GPU pueden ser no deterministas por defecto. En PyTorch activa `torch.use_deterministic_algorithms(True)`, asumiendo un coste moderado de rendimiento.



# Comparativa justa entre modelos

Comparar modelos solo tiene sentido si todos han sido evaluados bajo las **mismas condiciones experimentales**. Pequeñas diferencias en el conjunto de validación, la semilla o el preprocesamiento pueden hacer que un modelo inferior parezca superior.

### Mismo conjunto de evaluación

Usa exactamente el mismo split de test para todos los modelos comparados, generado con la misma semilla

### Mismas métricas

Define las métricas de éxito antes de entrenar, no después de ver los resultados

### Mismo preprocesamiento

Aplica exactamente el mismo pipeline de transformación, ajustado siempre sobre los mismos datos de entrenamiento

### Significancia estadística

Usa tests como Wilcoxon o bootstrap para determinar si las diferencias son reales o fruto del azar

# Conclusiones clave

1. **Registra siempre, aunque sea en un CSV**

   Cualquier registro sistemático es mejor que ninguno. La consistencia importa más que la herramienta.

2. **Versiona los tres pilares**

   Datos + código (commit) + modelo. Si falta uno, el experimento no es reproducible.

3. **Fija y registra la semilla aleatoria**

   Es el paso más sencillo y más ignorado para garantizar la reproducibilidad.

4. **Separa datos antes de preprocesar**

   Es la defensa más efectiva contra la fuga de datos silenciosa.

5. **Compara modelos bajo las mismas condiciones**

   Define métricas y splits antes de entrenar. Un experimento justo es la base de una buena decisión.

S15 · EN EL REPOSITORIO DEL CURSO

# Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

**S15_01_seguimiento_experimentos** — 50 min

Práctica de seguimiento de experimentos

sprints/S15/notebooks/S15_01_seguimiento_experimentos.ipynb

RETO

**Reto del sprint** — 130 min

Proyecto: modelo de referencia y primer modelo mejor. Anota todo en el registro de experimentos

sprints/S15/guia/Reto_Sprint_15.md